Ders 25 Çalışma Özeti

Ders 25 Çalışma Özeti

Hoca bu hafta hasta olduğu için dersi canlı anlatmadı. Önceki yıllardan birinin ders kaydını izletti. Özetteki anlatım o kayda aittir.

Genel Konular

- Alt dizgi arama (substring search) problemi ve kullanım alanları.
- Kaba kuvvet (brute force) arama ve geri dönüş alanlı (backup) sürümü.
- Horspool algoritması: kötü eşleşme tablosu (bad match table, shift table), BARBER ve TOOTH örnekleri.
- Boyer-Moore algoritması: kötü karakter ve iyi son ek (good suffix) kaydırmaları, BAOBAB örneği.
- Rabin-Karp algoritması: özet (hash) ile arama ve kayan özet (rolling hash) hesabı.
- Algoritmaların karşılaştırma tablosu.
- Final sınavı ve proje duyurusu.

Hocanın Özellikle Vurguladığı Kısımlar

- Problemde genellikle metin uzunluğu n, aranan desen (pattern) uzunluğu m'den çok büyüktür ($n \gg m$).
- Kaba kuvvette eşleşme olmayınca desen yalnız 1 karakter kaydırılır. Daha fazla kaydırılırsa bir eşleşme kaçırılabilir.
- Akan veride (stream) kaba kuvvet geri dönmek zorunda kalır. Bunun için m karakterlik bir geri dönüş alanı (backup) gerekir.
- Horspool, Boyer-Moore'un basitleştirilmiş hâlidir. İnternette ikisi sık sık karıştırılır:
  - Yalnız kötü eşleşme tablosu kullanan kod Horspool'dur.
  - Kötü eşleşme ve iyi son ek tablolarının ikisini birden kullanıp büyük olanı seçen kod Boyer-Moore'dur.
- Horspool desenin son karakterinden başlayarak sağdan sola karşılaştırır. Kaydırma miktarı her zaman desenin hizasındaki son metin karakterine göre tablodan okunur.
- Horspool basit gerçekleştirmesi ve hızı nedeniyle birçok arama kutusunda (search box) standarttır.
- Rabin-Karp'ta özet eşleşmesi yetmez. Çakışma (collision) olabileceği için eşleşen yerde m karakter tek tek kontrol edilir.
- Rabin-Karp'ta mod alınan asal sayı yeterince büyük seçilmelidir. Küçük seçilirse çakışma ve gereksiz kontrol artar.
- Hangi algoritmanın seçileceği bir ödünleşimdir (trade-off): hız, ek alan ve geri dönüş ihtiyacı birlikte düşünülür.

Kısa Tekrar Notları

- Kaba kuvvet: en kötü $O(mn)$, gerçek metinlerde tipik olarak yaklaşık $1.1n$.
- Horspool kaydırma tablosu:
  - Tüm karakterler için `table[c] = m`.
  - $j = 0 \ldots m-2$ için `table[P[j]] = m - 1 - j`. Sonraki tekrar öncekinin üzerine yazar.
  - Desenin son karakteri ($j = m-1$) tabloyu güncellemez.
- Horspool: en kötü $O(mn)$, en iyi yaklaşık n/m (alt doğrusal), ortalamada doğrusal. Ön işleme $O(m + |\Sigma|)$.

- Boyer-Moore:
  - $d_1 = \max(t_1(c) - k, 1)$. Burada k, sondan eşleşen karakter sayısıdır.
  - d_2: iyi son ek tablosundan gelen kaydırma.
  - $k = 0$ ise d_1 kadar kaydır. $k > 0$ ise $\max(d_1, d_2)$ kadar kaydır.
- Rabin-Karp: deseni ve her m uzunluklu metin penceresini $\bmod q$ ile özetle. Özetler eşitse karakterleri tek tek kontrol et.
- Kayan özet: $h_{i+1} = \left((h_i - t_i \cdot (10^{m-1} \bmod q)) \cdot 10 + t_{i+m}\right) \bmod q$.
- KMP soldan sağa çalışır ve geri dönüş istemez. Bu derste anlatılmadı.

Detaylı Açıklamalar

Alt Dizgi Arama Nerede Kullanılır?

- Web aramaları ve veritabanı sorguları.
- İntihal (plagiarism) tespiti: metin küçük parçalara bölünür, parçalar aranır.
- Adli bilişim (forensics): disk veya bellek imajında RSA anahtarı, `http`/`https` ile başlayan URL gibi imzaları aramak.
- Spam tespiti: "Viagra" gibi ifadeler ve sıfır/bir harfle yazılmış varyantları.
- Ağ trafiğinde belirli bir şüpheli deseni ("attack at dawn") aramak. Tüm trafiği okumadan yalnız deseni yakalayan bir makine.
- Bilgi çıkarımı (information extraction): HTML'de "Last Trade" ifadesini bulup ardından gelen `<b>` … `</b>` arasındaki sayıyı almak.

Kaba Kuvvet (Brute Force) Arama

```
int bruteForceSearch(char *text, char *pat) {
    int n = strlen(text), m = strlen(pat);
    for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m && text[i + j] == pat[j])
            j++;
        if (j == m)
            return i;          // eşleşme i indisinde
    }
    return -1;                 // bulunamadı
}
```

- Örnek: metin `ACAABC`, desen `AAB`. İlk denemede ikinci karakter tutmaz, 1 kaydırılır. 2 indisinde eşleşme bulunur.
- En kötü durum: metin `AAAA...AB`, desen `AAAAB`. Her konumda m karakter karşılaştırılır, sonuncusu tutmaz. Toplam yaklaşık mn karşılaştırma.
- Geri dönüş alanlı (backup) sürüm tek döngüyle yazılır:

```
int i, j;
for (i = 0, j = 0; i < n && j < m; i++) {
    if (text[i] == pat[j])
        j++;
    else {
        i -= j;                // geri dön (backup)
        j = 0;
    }
}
if (j == m) return i - m;      // bulundu
else return -1;                // bulunamadı
```

Horspool Algoritması

- Fikir: deseni sondan başa karşılaştır. Eşleşme bozulunca desen hizasındaki son metin karakterine bak:
  - Karakter desende hiç yoksa deseni m kadar kaydır.

  - Varsa, desendeki en sağdaki tekrarı (son karakter hariç) bu karaktere hizalanacak kadar kaydır.
- Durumlar:
  - Karakter desende yok: m kaydır.
  - Karakter desende var ama son karakter değil: en sağdaki tekrarına hizala.
  - Son karakter eşleşti ama deseninin geri kalanında tekrarı yok: m kaydır.
  - Son karakter eşleşti ve geri kalanında tekrarı var: o tekrara hizala.
- Bu kural bazen olabilecekten az kaydırır. Bu, Horspool'un zayıf noktasıdır; yine de kaba kuvvetten çok daha iyidir.

```
void shiftTable(char *pat, int table[], int size) {
    int m = strlen(pat);
    for (int i = 0; i < size; i++)
        table[i] = m;
    for (int j = 0; j <= m - 2; j++)
        table[(unsigned char)pat[j]] = m - 1 - j;
}

int horspool(char *text, char *pat, int table[]) {
    int n = strlen(text), m = strlen(pat);
    int i = m - 1;
    while (i <= n - 1) {
        int k = 0;
        while (k <= m - 1 && pat[m - 1 - k] == text[i - k])
            k++;
        if (k == m)
            return i - m + 1;  // eşleşme
        i += table[(unsigned char)text[i]];
    }
    return -1;
}
```

BARBER tablosu ve araması

Karakter	A	B	E	R	Diğerleri (boşluk dahil)
Kaydırma	4	2	1	3	6

- Metin: `JIM_SAW_ME_IN_A_BARBERSHOP`.

Deneme	Hizadaki son metin karakteri	Sonuç	Kaydırma
1	A	R ≠ A	4
2	E	R ≠ E	1
3	_	R ≠ _	6
4	B	R ≠ B	2
5	R	R tuttu, E ≠ A	3
6	R	Tüm karakterler tuttu	Bulundu (indis 16)

TOOTH tablosunun oluşması

j	Karakter	Değer ($5 - 1 - j$)	Tablonun son hâli
0	T	4	T = 4
1	O	3	O = 3
2	O	2	O = 2 (üzerine yazıldı)
3	T	1	T = 1 (üzerine yazıldı)

- H son karakter olduğu için döngüye girmez; H = 5. Diğer tüm karakterler 5.
- Metin `TRUSTHARDTOOTHBRUSHES` üzerinde: T → 1 kaydır, H tuttu, T tuttu, S ≠ O → H'ye göre 5 kaydır, O → 2 kaydır, T → 1 kaydır, sonra tüm karakterler tuttu.

Horspool'un en iyi ve en kötü durumu

- En kötü: metin tamamen `A`, desen `BAAAAA`. Her denemede m karşılaştırma yapılır ve `A`'nın değeri 1 olduğu için yalnız 1 kaydırılır. $O(mn)$.
- En iyi: her denemede ilk karşılaştırma tutmaz ve karakter desende yoktur. Her seferinde m kaydırılır. Yaklaşık n/m karşılaştırma.

Boyer-Moore Algoritması

- Horspool'dan önce (1977) ortaya çıktı. Horspool (1980) onun sadeleştirilmiş hâlidir.
- İki tablo kullanır:
  - Kötü karakter (bad symbol) tablosu: Horspool'un tablosu.
  - İyi son ek (good suffix) tablosu: sondan eşleşen k karakterlik son ekin desendeki başka bir tekrarına göre kaydırma.
- Kaydırma kararı:
  - $d_1 = \max(t_1(c) - k, 1)$. $t_1(c) - k$ sıfır veya negatif olabilir; desen geri çekilemeyeceği için en az 1 kaydırılır.
  - $k = 0$ ise yalnız d_1 kullanılır (Horspool gibi).
  - $k > 0$ ise $d = \max(d_1, d_2)$.
- Son ekin tekrarı yoksa deseni körü körüne m kaydırmak eşleşme kaçırabilir. Örneğin `ABCBAB` deseninde `BAB` tekrar etmiyor ama `AB` önek olarak başta var. Bu yüzden kaydırma, daha kısa son ekler için bulunan değerle sınırlanır.

ABCBAB için iyi son ek tablosu

k	Son ek	d_2
1	B	2
2	AB	4
3	BAB	4
4	CBAB	4
5	BCBAB	4

BAOBAB örneği

- Kötü karakter tablosu: A = 1, B = 2, O = 3, diğerleri 6.
- İyi son ek tablosu: $k = 1$ için 2, $k = 2 \ldots 5$ için 5.
- Metin: `BESS_KNEW_ABOUT_BAOBABS`.

Deneme	Karşılaştırma	k	d_1	d_2	Kaydırma
1	B ≠ K	0	6	-	6
2	B, A tuttu; B ≠ _	2	$6 - 2 = 4$	5	5
3	B tuttu; A ≠ _	1	$6 - 1 = 5$	2	5
4	Tüm karakterler tuttu	-	-	-	Bulundu

- Karmaşıklık: en kötü $O(mn)$. Gerçek metinlerde ortalamada doğrusal ve Horspool'dan biraz daha hızlı.

Rabin-Karp Algoritması

- Fikir: metni doğrudan karşılaştırmak yerine desenin bir imzasını (hash) çıkar ve metinde bu imzayı ara.
- Özet fonksiyonu tek yönlü bir dönüşümdür. Farklı girdiler aynı özeti verebilir (çakışma). Bu yüzden özet eşleşince karakterler kontrol edilir.
- Örnek: metin π'nin basamakları `3141592653589793`, desen `59265`, $q = 97$.

Pencere	Özet (mod97)	Desen özeti 95 ile
31415	84	Tutmadı
14159	94	Tutmadı
41592	76	Tutmadı
15926	18	Tutmadı
59265	95	Tuttu, karakterler de tuttu

- Kayan özet: her pencere için baştan mod almak gerekmez.
  - $14159 = (31415 - 3 \cdot 10000) \cdot 10 + 9$.
  - $10000 \bmod 97 = 9$ önceden hesaplanır.
  - $h = ((84 - 3 \cdot 9) \cdot 10 + 9) \bmod 97 = 579 \bmod 97 = 94$.
- Böylece her adım küçük sayılarla bir çıkarma, iki çarpma, bir toplama ve bir mod işlemidir.
- Karmaşıklık: ön işleme $O(m)$, ortalamada $O(m+n)$. Her özet eşleşmesi çakışma çıkarsa en kötü $O(mn)$.
- Aynı anda birden fazla desen aranacaksa özet tabanlı yöntemler avantajlıdır.

Karşılaştırma Tablosu

Algoritma	Garanti	Tipik	Geri dönüş	Ek alan
Kaba kuvvet	mn	$1.1n$	Gerekli	1
KMP (DFA)	$2n$	$1.1n$	Gerekmez	mR
KMP (mismatch geçişli)	$3n$	$1.1n$	Gerekmez	m
Boyer-Moore (yalnız kötü karakter)	mn	n/m	Gerekli	R
Rabin-Karp (Monte Carlo)	$7n$	$7n$	Gerekmez	1
Rabin-Karp (Las Vegas)	$7n$ (olasılıksal)	$7n$	Gerekli	1

- R: alfabe boyutu.
- Kullanım önerisi:
  - Metin kutularında arama: Boyer-Moore/Horspool.
  - Akan veri: KMP.
  - Çoklu desen: Rabin-Karp ve özet tabanlı türevleri.

Ders sonrası notlar

- **"Robin-Karp":** Algoritmanın adı Rabin-Karp'tır (Michael Rabin ve Richard Karp).
- **REORDER örneği:** Hoca kaydırmanın 4 olduğunu söyledi. REORDER'de son R hariç en sağdaki R, $j = 3$'tedir. Tablo değeri $7 - 1 - 3 = 3$'tür; desen 3 kaydırılır.
- **Kayan özet formülü:** Hoca formülü sözlü olarak "84 eksi 3 çarpı 9 çarpı 10" diye okudu. Doğru işlem sırası $((84-3\cdot9)\cdot10+9) \bmod 97$'dir. Parantez içi negatif çıkarsa q eklenerek pozitife çevrilir.
- **Asal sayı ve alfabe:** Hoca "alfabe 96'dan küçükse çakışma olmaz" dedi. Çakışma alfabe boyutuna değil, olası pencere sayısına bağlıdır. 5 basamaklı 10^5 farklı pencere 97 farklı özete düşer; çakışma kaçınılmazdır. Bu yüzden pratikte çok büyük bir asal seçilir.
- **Kaba kuvvette geri dönüş alanı:** Tabloda ek alan 1 yazıyor. Hoca da belirttiği gibi akan veride geri dönüş için m karakterlik alan gerekir.

Duyurular

- Final sınavı online olacak. 2 ya da 3 oturum olabilir: bir oturum kısa cevaplı sorular, ayrıca 1-2 algoritma veya kodlama sorusu.
- Proje, 19 Mayıs tatili ve hocanın hastalığı nedeniyle gecikti. En geç ertesi gün duyurulacak. Geç verildiği için beklentiler buna göre azaltılacak.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.